BIYAHEGO Midterm Project Documentation

Frontend Track for Butuan City Transportation

Item	Details
Student Name 1	Dailo
Student Name 2	Duro
Specialization	Frontend
Course and Section	[Insert course and section]
Instructor Name	[Insert instructor name]
Date of Submission	[Insert submission date]
Semester	A.Y. 2026-2027

This document presents the midterm frontend progress for BIYAHEGO, a Butuan City transportation system focused on helping commuters view routes, reserve seats, and track tickets through a responsive web interface. The current scope follows the midterm requirement: complete UI direction, core implemented screens, routing, reusable components, state management, mock API reading, validation, and initial testing.

I. Project Overview

BIYAHEGO is a transportation booking and route information frontend for Butuan City. The system addresses commuter difficulty in checking available local routes, comparing fares, and saving a trip reservation in a clear digital interface. The domain is transportation, with route examples covering Bancasi, Libertad, Langihan, Baan, CSU, Ampayon, Nasipit Port, and Masao Port.

Objectives

- Provide a clean route discovery interface for Butuan City transportation.
- Allow users to review fare, departure time, vehicle type, operator, and seat availability.
- Implement a validated booking form that saves passenger reservations to frontend state.
- Demonstrate initial frontend and backend agreement through a mock API contract.
- Apply responsive layout and accessibility basics for desktop and mobile users.

Target users

- Daily Butuan commuters who need fast access to route and fare information.
- Students traveling to Caraga State University and nearby areas.
- Visitors traveling to Bancasi Airport, Nasipit Port, or Masao Port.
- Dispatchers and transport operators who need an organized view of route status.

II. System Design and Development

A. System Architecture

The frontend is built as a React and Vite single page application. React was selected because it supports reusable components, organized state handling, and fast interface development. The application uses React Router for navigation and Redux Toolkit for route and booking state.

Item	Details
Framework	React with Vite
Routing	React Router with Dashboard, Routes, Book, and Tickets pages
State management	Redux Toolkit with route and booking slices
Mock API	public/mock-api/routes.json
Main reusable components	RouteMap, RouteCard, StatCard, navigation shell, form panels

Component and folder structure

- src/App.jsx contains the application layout, sidebar navigation, topbar, and route declarations.
- src/pages contains the Dashboard, Routes, Booking, and Tickets screens.
- src/components contains reusable visual pieces such as route cards, stats, and the Butuan route map.
- src/store contains Redux slices for route data and booking data.
- public/mock-api contains the JSON file used as a mock backend endpoint.

B. UI UX Design

The interface uses a practical dashboard style suitable for a transportation system. The main palette combines deep navy for structure, green for active transport status, amber for emphasis, white panels for readability, and light blue for the route map background. Typography uses a clean sans serif hierarchy so route names, status badges, and form labels are easy to scan.

The dashboard route visual was revised into a straight Butuan route corridor: Bancasi, Libertad, Langihan, Baan, CSU, and Ampayon. This makes the visualization easier to understand and more appropriate for explaining a city route during the presentation.

Responsiveness strategy

- Desktop uses a sidebar layout with content panels and multi-column cards.
- Tablet widths reduce card grids from four columns to two columns.
- Mobile widths collapse the sidebar into a menu button and stack all panels vertically.
- Buttons, forms, labels, and status text keep readable spacing on smaller screens.

C. Implementation Details Progress to Date

Feature	Implemented status	Notes
Dashboard	Done	Shows Butuan route corridor, summary cards, and commuter flow.
Routes page	Done	Reads route data from the mock JSON endpoint and displays route cards.
Booking page	Done	Includes passenger form, route selector, seat count, fare summary, validation, and error messages.
Tickets page	Done	Displays saved ticket data from Redux state.
State management	Done	Route selection, booking draft, and ticket list are stored in Redux.
API integration	Initial	Frontend reads from public/mock-api/routes.json. Backend endpoints can replace this later.
Backend integration	For final	Backend pair can provide matching REST endpoints for routes, bookings, and tickets.

The mock API currently provides route ID, origin, destination, vehicle type, departure, arrival, fare, available seats, status, and operator. This gives the backend pair a clear contract to follow when replacing the mock data with real endpoints.

Endpoint	Method	Purpose
/api/routes	GET	Return all available Butuan routes.
/api/routes/:id	GET	Return one route by route ID.
/api/bookings	POST	Create a new passenger booking.
/api/tickets/:passengerId	GET	Return tickets for one passenger.
/api/bookings/:id/status	PATCH	Update booking status after payment or check in.

D. Preliminary Testing and Debugging

Initial manual testing was completed through the browser and production build command. The app was checked for route loading, page navigation, form validation, successful booking save, and ticket display.

Test area	Result	Evidence
Production build	Passed	npm run build completed successfully.
Mock API display	Passed	Routes page displays four Butuan route records.
Booking validation	Passed	Empty submission shows required name and mobile number errors.

Valid booking	Passed	Valid passenger data is saved and appears in Tickets.
Responsive layout	Initial pass	Sidebar collapses to menu and cards stack on mobile widths.
Accessibility	Initial pass	Labels, semantic headings, focus outlines, and contrast-aware colors are included.

E. Shared ERD and API Agreement

The frontend and backend pairs should agree on the same data structure before full backend integration. The proposed ERD below is based on the BIYAHEGO flow from passenger route browsing to booking and ticket issuance.

Entity	Key attributes	Relationships
Passenger	passenger_id PK, full_name, mobile_number, email	One passenger can create many bookings.
Operator	operator_id PK, operator_name, contact_number	One operator can manage many vehicles.
Vehicle	vehicle_id PK, operator_id FK, plate_number, vehicle_type, capacity	One vehicle belongs to one operator and can be assigned to many trips.
Route	route_id PK, origin, destination, service_area, base_fare, estimated_duration	One route can have many scheduled trips.
Trip	trip_id PK, route_id FK, vehicle_id FK, departure_time, arrival_time, status	One trip belongs to one route and one vehicle.
Booking	booking_id PK, passenger_id FK, trip_id FK, seat_count, total_amount, booking_status, created_at	One booking belongs to one passenger and one trip.
Ticket	ticket_id PK, booking_id FK, ticket_code, issued_at, ticket_status	One ticket is issued from one booking.

For the midterm, the frontend uses mock route records with the same fields expected from the future GET /api/routes endpoint. For the final, the backend pair can return the same JSON shape from a real database so the frontend can replace the mock URL with the backend URL.

III. Ethical and Professional Considerations

BIYAHEGO avoids unnecessary personal data collection in the current frontend. The booking form asks only for passenger name, mobile number, route, and seat count because these are needed to create a trip reservation. Input validation helps prevent invalid data from being saved.

Accessibility is considered through readable color contrast, clear headings, labeled inputs, keyboard focus styles, responsive layout, and concise status messages. Icons come from the Lucide React library, and the current visual design uses custom CSS rather than copyrighted transport images.

IV. Midterm Progress Report

The frontend pair has completed the core interface needed for the midterm demo. The application runs locally, includes multiple screens, uses state management, reads mock API data, and demonstrates a validated booking flow.

Feature or screen	Status	Notes
Dashboard	Done	Butuan route corridor and summary cards are implemented.

Routes	Done	Displays route cards from mock API data.
Booking	Done	Validated form and fare summary are implemented.
Tickets	Done	Saved tickets are displayed from Redux state.
Documentation	In progress	This draft can be finalized with student details and screenshots.
GitHub repository	Pending	Create repository and commit history before presentation.
Backend connection	Pending for final	Backend pair must implement agreed endpoints.

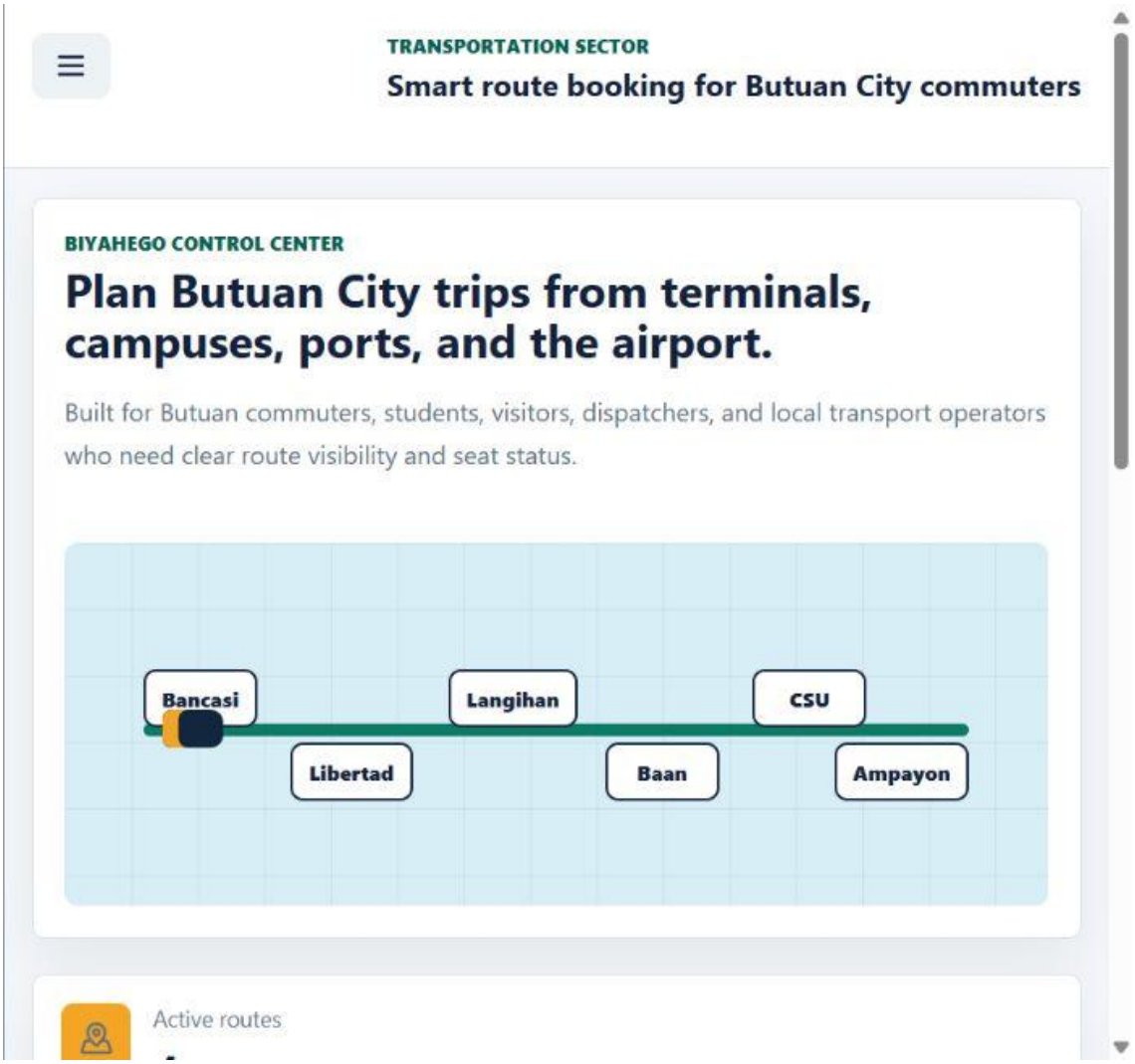


Figure 1 Dashboard with Butuan route corridor.

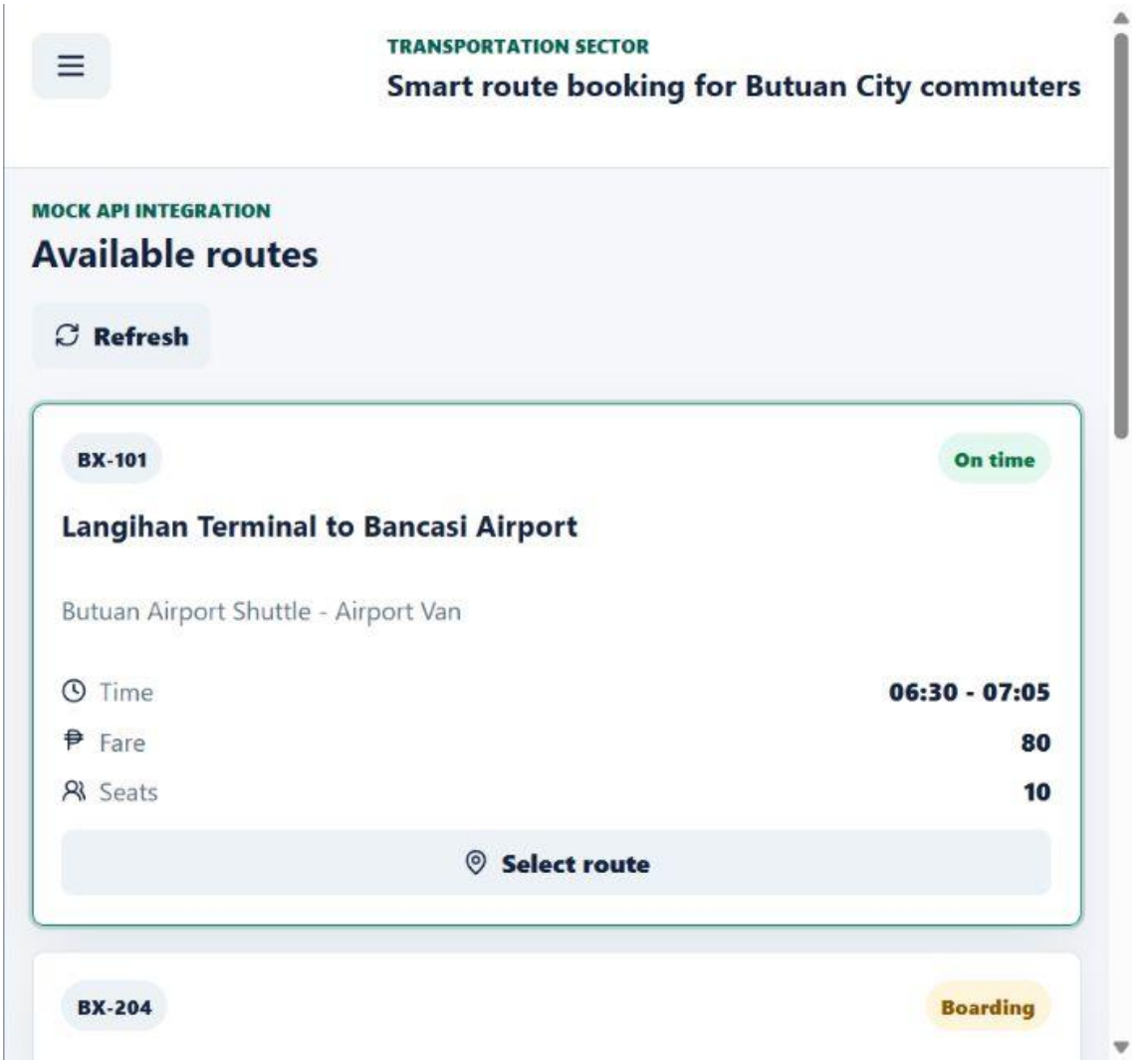


Figure 2 Routes page showing Butuan transportation records.

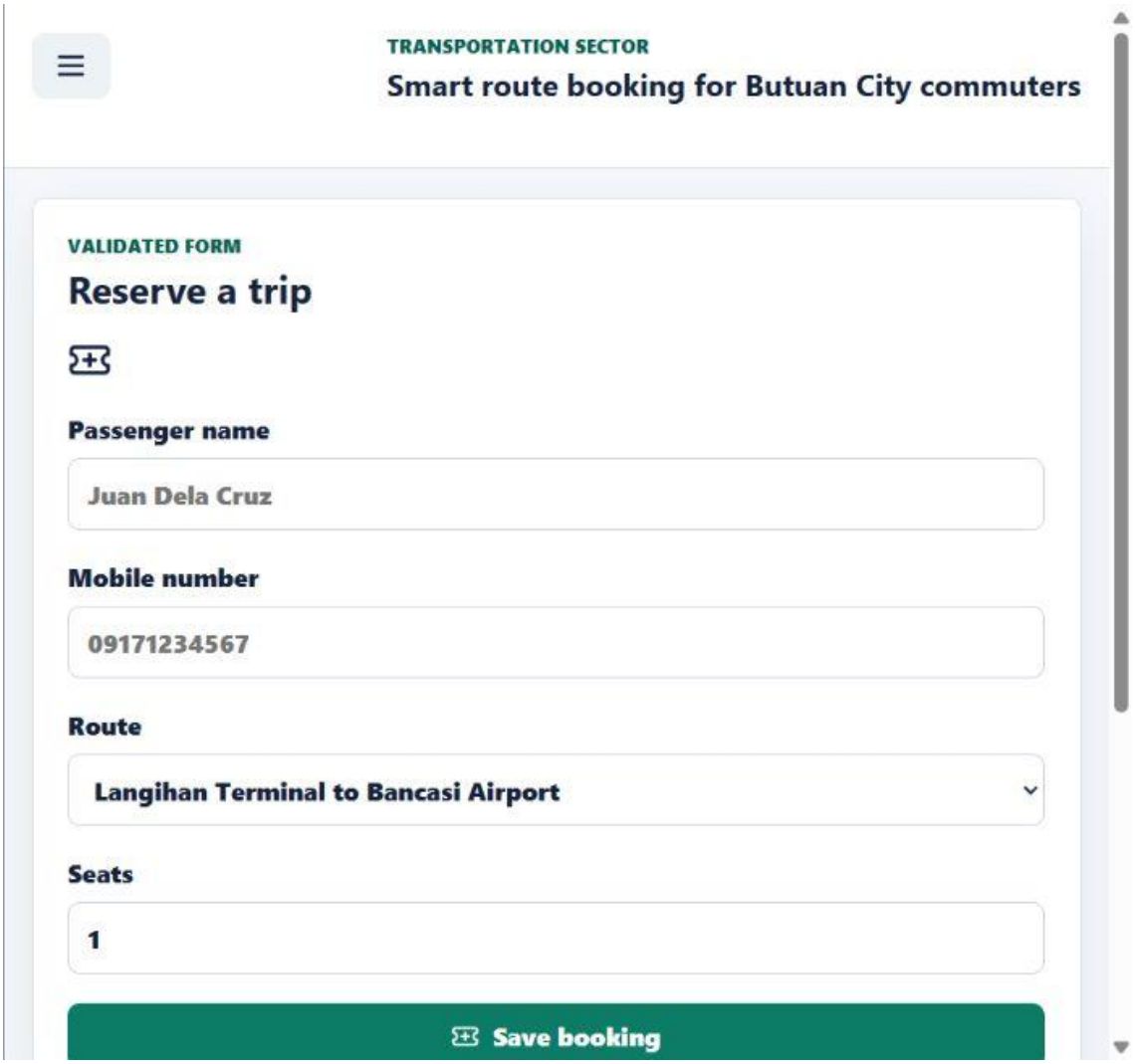


Figure 3 Booking page with validation and fare summary.

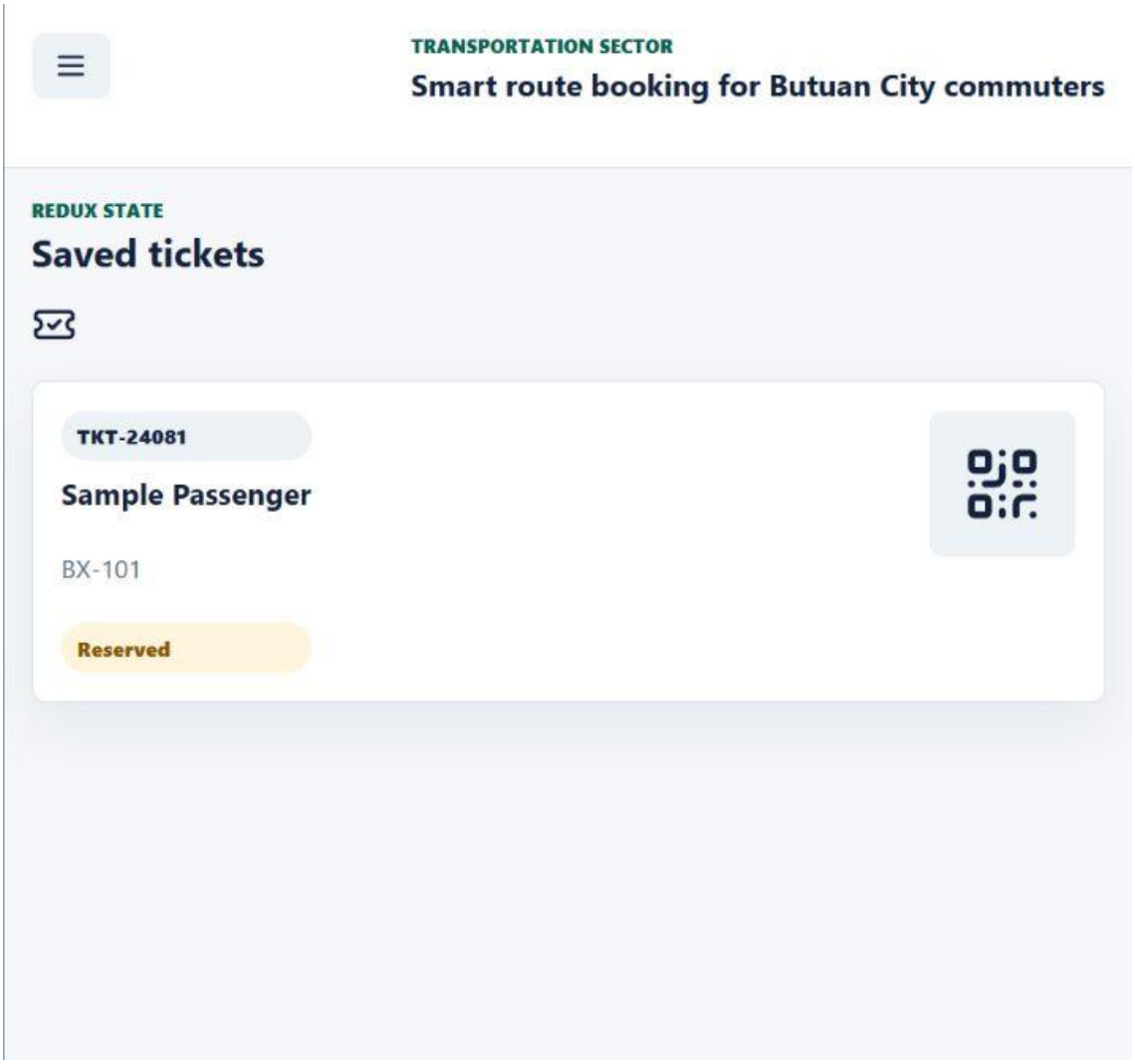


Figure 4 Tickets page showing saved frontend state.

Challenges encountered

- The first route visual looked too generic, so it was revised into a straight Butuan route line.
- The frontend needed a clear mock API structure so the backend pair can later match the same fields.
- The booking form needed validation messages that are visible and easy for users to understand.

Team contributions

Member	Contribution	Status
Dailo	Frontend layout, route pages, route map refinement, and documentation content review.	To confirm
Duro	Booking form, ticket state flow, mock API review, and responsive checks.	To confirm

V. Remaining Work and Plan for Final

For the final period, the frontend should be polished further and integrated with the backend pair. The current structure already prepares the application for real API endpoints, expanded validation, and deployment.

Task	Responsible member	Target date

Replace mock API with backend /api/routes endpoint.	Frontend and backend pairs	[Insert date]
Connect POST /api/bookings to the booking form.	Frontend and backend pairs	[Insert date]
Add login or passenger profile flow if required.	Frontend pair	[Insert date]
Improve ticket view with QR or ticket code from backend.	Frontend pair	[Insert date]
Complete browser and mobile responsiveness testing.	Frontend pair	[Insert date]
Prepare final screenshots and deployed demo link.	Frontend pair	[Insert date]
Write final evaluation, conclusion, and recommendations.	All members	[Insert date]

Appendix

- GitHub repository: [Insert repository link]
- Design file or wireframes: [Insert Figma or design link if available]
- Mock API source: public/mock-api/routes.json
- Local demo command: npm run dev
- Local demo URL: http://127.0.0.1:5173/